

RELATÓRIO DE INTEGRAÇÃO MOM

Sprint 2 - FastDelivery com RabbitMQ

Disciplina: Laboratório de Desenvolvimento de Aplicações Móveis e Distribuídas

Projeto: FastDelivery

Referência: seção 3.3, página 5 do enunciado Projeto_LAMD_60445_2026_1

Escopo: integração assíncrona orientada a eventos com RabbitMQ

Objetivo

A Sprint 2 integrou um Middleware Orientado a Mensagens (MOM) ao backend Flask do FastDelivery. O objetivo foi permitir que fatos relevantes do domínio de entregas fossem publicados e processados de forma assíncrona, sem chamada REST direta entre produtor e consumidor.

Decisões de design

Foi adotado o RabbitMQ por ser um broker dedicado, com suporte nativo a filas duráveis, mensagens persistentes, confirmação de publicação, confirmação manual de consumo e Dead-Letter Queue (DLQ). O padrão aplicado é Publish-Subscribe sobre a exchange topic **fastdelivery.events**.

O backend Flask atua como produtor. Por padrão, o consumidor roda em outro processo com **py consumer_worker.py**. Assim, o produtor publica mesmo quando o consumidor está desligado; a fila **fastdelivery.entregas** armazena o backlog; quando o worker inicia, ele consome as mensagens pendentes.

Fluxo implementado

1. **POST /entregas** cria uma entrega e publica **entrega.criada**.
2. **PATCH /entregas/<id>/status** altera o estado e publica **entrega.status_atualizado**.

Os eventos seguem pela exchange **fastdelivery.events** para a fila durável **fastdelivery.entregas**. O worker processa a mensagem e somente então envia ack. Falhas recebem nack(requeue=False) e seguem para **fastdelivery.dlq**.

Desafios e conclusão

A primeira versão utilizava Redis. A revisão técnica motivou a migração para RabbitMQ e a inclusão de mensagens persistentes (`delivery_mode=2`), publisher confirms, uma tentativa de reconexão, DLQ e histórico idempotente no SQLite por `evento_id`. A solução atende ao enunciado: o broker é o ponto central, existem dois eventos de negócio documentados e o fluxo assíncrono pode ser demonstrado por logs, UI do RabbitMQ e GET /eventos.

Anexo 1 - Conferência do enunciado

A seção 3.3 solicita MOM operacional com evidência, produtor e consumidor, publicação em dois momentos de negócio, catálogo dos eventos, prova de assincronicidade e relatório de integração de uma página. A página anterior é o relatório principal; os anexos apoiam a demonstração.

Requisito	Implementação	Evidência recomendada
MOM operacional	RabbitMQ via Docker Compose, AMQP 5672 e UI 15672	UI e docker compose ps
Produtor e consumidor	EntregaEventProducer e consumer_worker.py	Terminais separados
Dois momentos	Criação e alteração de status	Logs dos dois tópicos
Catálogo	Eventos, routing keys e payloads	Anexo 2
Assincronicidade	Backlog com worker desligado	Ready > 0 e depois Ready = 0
Relatório	Síntese de decisões e desafios	Primeira página

O que foi adicionado

Componente	Responsabilidade
app/mom/barramento.py	Interface EventBus, RabbitMQEventBus e InMemoryEventBus para testes
app/mom/eventos.py	Tópicos, payload comum e evento_id
app/mom/producer.py	Publicação dos fatos do domínio
app/mom/consumer.py	Handlers, histórico persistente e deduplicação
consumer_worker.py	Consumidor standalone sem Flask ou REST
evento_processado_repository.py	Histórico compartilhado no SQLite
docker-compose.yml	RabbitMQ com UI e volume persistente
GET /eventos	Consulta aos eventos processados

Garantias técnicas

- Exchange topic fastdelivery.events e fila durável fastdelivery.entregas ligada com routing key #.
- delivery_mode=2, publisher confirms e mandatory=True.
- Ack manual após sucesso; nack(requeue=False) envia falhas para fastdelivery.dlq.
- Nova tentativa após falha de conexão e deduplicação por evento_id no SQLite.

Anexo 2 - Catálogo de eventos

entrega.criada

Campo	Valor
Momento	Após EntregaUseCases.criar_entrega() persistir a nova entrega
Produtor	EntregaEventProducer.entrega_criada()
Consumidor	_ao_criar_entrega() executado por consumer_worker.py
Exchange / routing key	fastdelivery.events / entrega.criada
Fila	fastdelivery.entregas

```
{
  "evento_id": "uuid-gerado-na-publicacao",
  "evento": "entrega.criada",
  "dados": {
    "id": 1, "descricao": "Pacote Sprint 2",
    "origem": "Rua A, 100", "destino": "Rua B, 200",
    "status": "pendente", "cliente_id": "cliente-001"
  },
  "timestamp": "2026-06-01T10:00:00"
}
```

entrega.status_atualizado

Campo	Valor
Momento	Após EntregaUseCases.atualizar_status() persistir o novo estado
Produtor	EntregaEventProducer.status_alterado()
Consumidor	_ao_alterar_status() executado por consumer_worker.py
Exchange / routing key	fastdelivery.events / entrega.status_atualizado
Fila	fastdelivery.entregas

```
{
  "evento_id": "uuid-gerado-na-publicacao",
  "evento": "entrega.status_atualizado",
  "dados": {
    "id": 1, "status_anterior": "pendente",
    "status_novo": "aceito", "cliente_id": "cliente-001"
  },
  "timestamp": "2026-06-01T10:02:00"
}
```

Anexo 3 - Testes executados

Comando executado em **Code/server/**:

```
py -m unittest discover -s tests -v
```

Resultado obtido em 01/06/2026:

```
Ran 8 tests in 0.087s  
OK
```

Teste	Validação
Modo em memória explícito	EVENT_BUS=in_memory cria o barramento reservado a testes
Configuração padrão	Sem variável específica, cria RabbitMQEventBus
Ack de sucesso	Handler bem-sucedido produz basic_ack
Falha de handler	Exceção produz basic_nack(requeue=False) para DLQ
Tópico desconhecido	Mensagem sem handler também segue para DLQ
Canal encerrado	A conexão de publicação é substituída
Falha transitória	O produtor repete a publicação uma vez
Idempotência	O mesmo evento_id é persistido uma única vez

Nota: `tests/test_entregas.py` ainda é um placeholder. Os endpoints REST devem ser comprovados manualmente no roteiro a seguir. Não apresente a suíte MOM como se também cobrisse endpoints.

Anexo 4 - Roteiro de execução e prints

1. Preparar o broker

Abra o Docker Desktop. Depois execute em um PowerShell:

```
cd C:\Users\gpercope\Documents\GitHub\Lab-DAMD\Code\server
docker compose up -d
docker compose ps
Copy-Item .env.example .env -ErrorAction SilentlyContinue
pip install -r requirements.txt
```

Abra **http://localhost:15672** e entre com **guest / guest**.

Nota: Print E1: capture docker compose ps e, na UI, Queues and Streams com fastdelivery.entregas e fastdelivery.dlq.

2. Subir apenas o backend produtor

```
cd C:\Users\gpercope\Documents\GitHub\Lab-DAMD\Code\server
py main.py
```

Nota: Print E2: capture o terminal do Flask mostrando Mecanismo MOM.: rabbitmq e Consumidor in-process: OFF.

3. Publicar com o worker desligado

```
$body = @{
    descricao = "Pacote demonstracao Sprint 2"
    origem = "Rua A, 100"
    destino = "Rua B, 200"
    cliente_id = "cliente-001"
} | ConvertTo-Json
```

```
$entrega = Invoke-RestMethod -Method Post `
    -Uri "http://localhost:5000/entregas" `
    -ContentType "application/json" -Body $body
```

```
Invoke-RestMethod -Method Patch `
    -Uri "http://localhost:5000/entregas/${$entrega.id}/status" `
    -ContentType "application/json" -Body '{"status":"aceito"}
```

Nota: Print E3: com o worker desligado, abra Queues and Streams > fastdelivery.entregas e capture Ready aumentando em 2. Este é o print principal da assincronicidade.

Anexo 4 - Roteiro de execução e prints (continuação)

4. Ligar o consumidor independente

```
cd C:\Users\gpercope\Documents\GitHub\Lab-DAMD\Code\server
py consumer_worker.py
```

Nota: Print E4: capture os logs do worker para entrega.criada e entrega.status_atualizado. **Print E5:** atualize a UI e capture Ready = 0.

5. Consultar o histórico

```
Invoke-WebRequest -Uri "http://localhost:5000/eventos" |
ConvertTo-Json -Depth 10
```

Nota: Print E6: capture o JSON contendo os dois eventos, seus evento_id, tipos, mensagens e payloads.

6. Executar a suíte automatizada

```
py -m unittest discover -s tests -v
```

Nota: Print E7: capture a saída final com Ran 8 tests e OK.

Evidências extras

Persistência: com o worker desligado, publique uma entrega, confira Ready > 0 e execute:

```
docker compose restart rabbitmq
docker compose ps
```

Após o container ficar saudável, a mensagem deve continuar na fila. Capture antes e depois.

DLQ: com o worker ligado, publique um tópico sem handler:

```
$codigo = @'
from app.mom.barramento import criar_barramento
from app.mom.eventos import obter_payload
bus = criar_barramento()
bus.publicar('entrega.desconhecida',
            obter_payload('entrega.desconhecida', {'teste': True}))
bus.parar()
'@
py -c $codigo
```

Capture o log do worker e a fila **fastdelivery.dlq** com mensagem em Ready.

Anexo 5 - Roteiro curto para apresentação

1. Mostrar a UI do RabbitMQ e as filas fastdelivery.entregas e fastdelivery.dlq.
2. Explicar que o Flask é produtor puro e que consumer_worker.py roda separado, sem Flask e sem REST.
3. Manter o worker desligado, executar POST e PATCH e mostrar o backlog crescendo.
4. Ligar o worker e mostrar os dois eventos sendo consumidos.
5. Executar GET /eventos e mostrar o histórico persistente.
6. Mostrar rapidamente a saída dos oito testes automatizados.
7. Como evidência extra, demonstrar a DLQ ou a persistência após reinício do RabbitMQ.

Aviso sobre evidências antigas

Nota: Os arquivos em **docs/Sprint2/evidencias** foram gerados antes da migração e ainda descrevem Redis. Não use esses registros como prova atual da solução RabbitMQ. Produza capturas novas com o roteiro deste documento.